Guia de funcionamento e lógica de programação

Sistema de controle de estoque para um hortifrúti

Material de estudo baseado no pseudocódigo do projeto, com explicações, exemplos de pseudocódigo e trechos equivalentes em Java.

Funcionalidades: cadastrar, listar, consultar, editar e remover produtos; calcular o valor total do estoque.

Os exemplos Java são didáticos. Eles mantêm o limite de 100 produtos e o armazenamento em memória, mas incluem alguns tratamentos de entrada que não estão detalhados no pseudocódigo original.

Sumário

- 1. Objetivo e visão geral
- 2. Vetores e armazenamento dos dados
- 3. Variáveis e estruturas de programação
- 4. Algoritmo principal e menu
- 5. ler_categoria()
- 6. buscar_produto()
- 7. cadastrar_produto()
- 8. listar_produtos()
- 9. consultar_produto()
- 10. editar_produto()
- 11. remover_produto()
- 12. calcular_valor_estoque()
- 13. Exemplo de execução
- 14. Validações, limitações e melhorias
- 15. Exemplo Java integrado

O pseudocódigo usa posições numeradas de 1 a TOTAL_PRODUTOS. Em Java, os índices dos vetores começam em 0; por isso, os exemplos Java ajustam os limites dos laços.

1. Objetivo e visão geral

O sistema organiza os dados básicos dos produtos de um hortifrúti. Cada produto possui nome, categoria, preço unitário e quantidade em estoque. O usuário escolhe uma operação em um menu numérico.

Opção	Operação	O que faz
1	Cadastrar	Adiciona produto após validar os dados.
2	Listar	Exibe todos os produtos cadastrados.
3	Consultar	Localiza um produto pelo nome e mostra seus dados.
4	Editar	Atualiza os dados de um produto existente.
5	Remover	Exclui um produto e reorganiza os vetores.
6	Calcular estoque	Soma preço × quantidade de cada produto.
0	Sair	Encerra o menu.

Visão geral do algoritmo

```
TOTAL_PRODUTOS <- 0
REPITA
    mostrar menu
    LEIA OP
    chamar método conforme OP
ATÉ OP = 0
mostrar "Sistema encerrado."
```

Cada método possui uma responsabilidade específica. Essa divisão facilita entender, testar e corrigir o programa. Os dados não são gravados em arquivo ou banco de dados: ao encerrar o programa, eles são perdidos.

2. Vetores e armazenamento dos dados

O projeto usa quatro vetores paralelos. Cada índice representa um produto, e os dados que pertencem ao mesmo produto ocupam a mesma posição em todos os vetores.

```
NOMES[100]       // texto: nome
CATEGORIAS[100]  // texto: Fruta, Legume ou Verdura
PRECOS[100]      // numérico: preço unitário
QUANTIDADES[100] // numérico: unidades em estoque
```

Índice	NOMES	CATEGORIAS	PRECOS	QUANTIDADES
1	Maçã	Fruta	2,50	10
2	Cenoura	Legume	3,00	5
3	Alface	Verdura	2,00	8

Por exemplo, os quatro valores da Maçã estão todos na posição 1 do pseudocódigo. A variável TOTAL_PRODUTOS informa quantos lugares estão ocupados.

Equivalente em Java

```
static String[] nomes = new String[100];
static String[] categorias = new String[100];
static double[] precos = new double[100];
static int[] quantidades = new int[100];
static int totalProdutos = 0;
```

Java começa a contar os índices em 0. Assim, a primeira posição é nomes[0], e os produtos cadastrados ficam nos índices de 0 até totalProdutos - 1.

3. Variáveis e estruturas de programação

Elemento	Papel no sistema
TOTAL_PRODUTOS	Quantidade de produtos cadastrados.
OP	Opção digitada no menu.
I	Índice usado para percorrer vetores.
POSICAO	Posição em que um produto foi encontrado.
-1	Indica que a busca não encontrou o produto.
SE / SENÃO	Executa caminhos diferentes conforme uma condição.
REPITA...ATÉ	Repete até uma condição ser satisfeita.
PARA	Percorre uma sequência de posições.
RETORNE	Devolve um resultado ao método que fez a chamada.

Exemplo de validação

```
REPITA
    LEIA PRECO
    SE PRECO < 0
        ESCREVA "O preço não pode ser negativo."
ATÉ PRECO >= 0
```

O programa pede o preço novamente enquanto ele for negativo. Preço zero é permitido. A mesma lógica é usada para a quantidade.

Exemplo Java

```
while (preco < 0) {
    System.out.println("Preço inválido.");
    preco = sc.nextDouble();
}
```

Este trecho mostra a ideia da repetição. Em um programa completo, também é necessário cuidar da leitura de texto e de entradas que não sejam números.

4. Algoritmo principal e menu

O algoritmo principal inicializa TOTAL_PRODUTOS com zero, apresenta as opções, lê a escolha e chama o método correspondente. O menu continua até o usuário escolher 0.

```
SE OP = 1 ENTÃO cadastrar_produto()
SENÃO SE OP = 2 ENTÃO listar_produtos()
SENÃO SE OP = 3 ENTÃO consultar_produto()
SENÃO SE OP = 4 ENTÃO editar_produto()
SENÃO SE OP = 5 ENTÃO remover_produto()
SENÃO SE OP = 6 ENTÃO calcular_valor_estoque()
SENÃO SE OP <> 0 ENTÃO mostrar "Opção inválida!"
```

Exemplo Java

```
switch (op) {
    case 1 -> cadastrarProduto();
    case 2 -> listarProdutos();
    case 3 -> consultarProduto();
    case 4 -> editarProduto();
    case 5 -> removerProduto();
    case 6 -> calcularValorEstoque();
    case 0 -> System.out.println("Sistema encerrado.");
    default -> System.out.println("Opção inválida!");
}
```

O switch escolhe a operação. Para repetir o menu, ele fica dentro de um laço, por exemplo do...while (op != 0). A opção inválida é qualquer valor diferente das opções previstas e diferente de zero.

5. Método ler_categoria()

Apresenta as opções Fruta, Legume e Verdura. Se a opção digitada não for 1, 2 ou 3, informa o erro e pergunta novamente. Quando válida, devolve o nome da categoria.

```
REPITA
    mostrar 1 - Fruta; 2 - Legume; 3 - Verdura
    LEIA OPCAO
    se 1: CATEGORIA <- "Fruta"
    se 2: CATEGORIA <- "Legume"
    se 3: CATEGORIA <- "Verdura"
    caso contrário: mostrar erro
ATÉ CATEGORIA <> ""
RETORNE CATEGORIA
```

Exemplo Java

```
static String lerCategoria(Scanner sc) {
    while (true) {
        System.out.println("1 - Fruta\n2 - Legume\n3 - Verdura");
        String op = sc.nextLine().trim();
        switch (op) {
            case "1": return "Fruta";
            case "2": return "Legume";
            case "3": return "Verdura";
            default: System.out.println("Categoria inválida.");
        }
    }
}
```

O while repete a pergunta. O return encerra o método e entrega a categoria escolhida ao método que o chamou.

6. Método buscar_produto()

Percorre os nomes cadastrados e compara cada um com o nome procurado. Se encontrar, devolve a posição. Se terminar a busca sem encontrar, devolve -1.

```
PARA I <- 1 ATÉ TOTAL_PRODUTOS
    SE NOMES[I] = NOME_PROCURADO
        RETORNE I
FIM_PARA
RETORNE -1
```

Exemplo Java

```
static int buscarProduto(String nomeProcurado) {
    for (int i = 0; i < totalProdutos; i++) {
        if (nomes[i].equalsIgnoreCase(nomeProcurado)) {
            return i;
        }
    }
    return -1;
}
```

O exemplo Java usa equalsIgnoreCase para ignorar diferenças entre maiúsculas e minúsculas; isso é uma escolha do exemplo. O pseudocódigo original não define essa regra explicitamente. Lembre-se também de que a posição retornada em Java começa em 0.

7. Método cadastrar_produto()

Cadastra um produto novo. Primeiro verifica se há espaço no vetor. Depois lê o nome e impede duplicidade. Em seguida solicita categoria, preço e quantidade, repetindo as entradas numéricas negativas. Por fim, grava os quatro dados na mesma posição.

- Se já existem 100 produtos, não permite novo cadastro.
- Se o nome já existe, informa a duplicidade e não cadastra.
- A categoria é obtida por ler_categoria().
- Preço e quantidade devem ser maiores ou iguais a zero.
- Ao concluir, incrementa o contador e armazena os dados.

```
SE TOTAL_PRODUTOS = 100: mostrar limite atingido
SENÃO:
    ler NOME
    POSICAO <- buscar_produto(NOME)
    se POSICAO <> -1: mostrar nome duplicado
    senão:
        CATEGORIA <- ler_categoria()
        ler PRECO até PRECO >= 0
        ler QUANTIDADE até QUANTIDADE >= 0
        TOTAL_PRODUTOS <- TOTAL_PRODUTOS + 1
        gravar os quatro valores na nova posição
```

Exemplo Java

```
int pos = totalProdutos;
nomes[pos] = nome;
categorias[pos] = categoria;
precos[pos] = preco;
quantidades[pos] = quantidade;
totalProdutos++;
```

Se totalProdutos vale 0, o primeiro produto é gravado no índice 0. Depois o contador passa a 1. O trecho de gravação deve ser executado somente depois das validações.

8. Método listar_produtos()

Se não houver produtos, mostra uma mensagem. Caso contrário, percorre todos os produtos cadastrados e exibe nome, categoria, preço e quantidade.

```
SE TOTAL_PRODUTOS = 0
    ESCREVA "Não há produtos cadastrados."
SENÃO
    PARA I <- 1 ATÉ TOTAL_PRODUTOS
        mostrar os quatro campos do produto
    FIM_PARA
```

Exemplo Java

```
static void listarProdutos() {
    if (totalProdutos == 0) {
        System.out.println("Não há produtos cadastrados.");
        return;
    }
    for (int i = 0; i < totalProdutos; i++) {
        System.out.println("Nome: " + nomes[i]);
        System.out.println("Categoria: " + categorias[i]);
        System.out.printf("Preço: R$ %.2f%n", precos[i]);
        System.out.println("Quantidade: " + quantidades[i]);
    }
}
```

O laço percorre apenas os índices ocupados. O formato %.2f exibe o preço com duas casas decimais.

9. Método consultar_produto()

Solicita o nome, chama buscar_produto() e verifica o resultado. Se receber -1, informa que não encontrou. Caso contrário, usa a posição retornada para mostrar os quatro dados.

```
LEIA NOME
POSICAO <- buscar_produto(NOME)
SE POSICAO = -1
    mostrar "Produto não encontrado."
SENÃO
    mostrar nome, categoria, preço e quantidade
```

Exemplo Java

```
int pos = buscarProduto(nome);
if (pos == -1) {
    System.out.println("Produto não encontrado.");
} else {
    System.out.println(nomes[pos]);
    System.out.println(categorias[pos]);
    System.out.println(precos[pos]);
    System.out.println(quantidades[pos]);
}
```

A consulta reutiliza a busca em vez de implementar novamente a lógica de procura. Isso evita repetição de código.

10. Método editar_produto()

Localiza o produto original e solicita os novos dados. O novo nome pode ser igual ao nome atual, mas não pode pertencer a outro produto. Depois de validar categoria, preço e quantidade, substitui os valores na posição original.

```
POSICAO <- buscar_produto(NOME)
se POSICAO = -1: mostrar "Produto não encontrado."
senão:
    ler NOVO_NOME
    POSICAO_NOME <- buscar_produto(NOVO_NOME)
    se POSICAO_NOME <> -1 E POSICAO_NOME <> POSICAO:
        mostrar "Já existe outro produto com esse nome."
    senão:
        ler categoria, preço e quantidade válidos
        atualizar os quatro vetores na POSICAO
```

Exemplo Java

```
int posNome = buscarProduto(novoNome);
if (posNome != -1 && posNome != pos) {
    System.out.println("Já existe outro produto com esse nome.");
    return;
}
nomes[pos] = novoNome;
categorias[pos] = novaCategoria;
precos[pos] = novoPreco;
quantidades[pos] = novaQuantidade;
```

A comparação posNome != pos permite manter o próprio nome, mas impede usar o nome de outro produto. As atribuições finais devem ocorrer depois de validar todos os novos dados.

11. Método remover_produto()

Procura o produto pelo nome. Se ele existir, desloca os elementos seguintes uma posição para trás, copiando nome, categoria, preço e quantidade juntos. Ao final, reduz TOTAL_PRODUTOS em um.

```
POSICAO <- buscar_produto(NOME)
SE POSICAO = -1: mostrar "Produto não encontrado."
SENÃO:
    PARA I <- POSICAO ATÉ TOTAL_PRODUTOS - 1
        campo[I] <- campo[I + 1] // para cada vetor
    TOTAL_PRODUTOS <- TOTAL_PRODUTOS - 1
```

Exemplo Java

```
for (int i = pos; i < totalProdutos - 1; i++) {
    nomes[i] = nomes[i + 1];
    categorias[i] = categorias[i + 1];
    precos[i] = precos[i + 1];
    quantidades[i] = quantidades[i + 1];
}
totalProdutos--;
```

Exemplo: se houver Maçã, Cenoura e Alface e Cenoura for removida, Alface ocupa a posição anterior. O contador passa de 3 para 2. O valor antigo da última posição pode continuar no array, mas não é considerado cadastrado.

12. Método calcular_valor_estoque()

Começa com o total igual a zero. Para cada produto, multiplica o preço pela quantidade e adiciona o subtotal ao total geral. Por fim, exibe o valor do estoque.

```
VALOR_TOTAL <- 0
PARA I <- 1 ATÉ TOTAL_PRODUTOS
    VALOR_TOTAL <- VALOR_TOTAL + (PRECOS[I] * QUANTIDADES[I])
FIM_PARA
mostrar VALOR_TOTAL
```

Exemplo Java

```
double valorTotal = 0;
for (int i = 0; i < totalProdutos; i++) {
    valorTotal += precos[i] * quantidades[i];
}
System.out.printf("Valor total: R$ %.2f%n", valorTotal);
```

Produto	Preço	Quantidade	Subtotal
Maçã	R$ 2,50	10	R$ 25,00
Cenoura	R$ 3,00	5	R$ 15,00
Alface	R$ 2,00	8	R$ 16,00
Total			R$ 56,00

Cálculo: (2,50 × 10) + (3,00 × 5) + (2,00 × 8) = R$ 56,00. Esse é o valor estimado dos itens armazenados, não o lucro do estabelecimento.

13. Exemplo de execução

Passo	Ação	Resultado
1	Escolher 1	O sistema inicia o cadastro.
2	Informar Maçã, Fruta, R$ 2,50 e 10 unidades	O produto é guardado.
3	Escolher 2	A listagem mostra Maçã.
4	Escolher 3 e digitar Maçã	A consulta exibe seus dados.
5	Editar a quantidade para 12	O produto passa a ter 12 unidades.
6	Escolher 6	Se só houver Maçã, o valor é R$ 30,00.
7	Escolher 0	O programa encerra.

Os métodos se complementam: cadastrar armazena, listar e consultar exibem, editar altera, remover exclui e calcular usa os dados cadastrados.

14. Validações, limitações e melhorias

Validações previstas

• Máximo de 100 produtos.

• Não permite nomes duplicados no cadastro.

• Não permite renomear um produto para o nome de outro já cadastrado.

• Categoria limitada a Fruta, Legume ou Verdura.

• Preço e quantidade não podem ser negativos.

• Mensagens para produto não encontrado e opção inválida.

Limitações atuais

• Os dados ficam em memória e se perdem ao fechar o programa.

• O pseudocódigo não detalha o que fazer quando o usuário digita texto em campos numéricos.

• A comparação de nomes não especifica tratamento de maiúsculas/minúsculas ou espaços.

• double é adequado para um exemplo introdutório; sistemas financeiros costumam usar BigDecimal para dinheiro.

• Não há persistência, usuários, fornecedores, validade ou histórico de movimentações.

Melhorias futuras

- Salvar os dados em arquivo ou banco de dados.
- Tratar todas as entradas inválidas sem interromper o programa.
- Padronizar espaços e comparação de nomes.
- Criar uma classe Produto para agrupar os dados.
- Testar remoção do primeiro e do último produto, estoque vazio e limite de 100 produtos.

As melhorias são sugestões para versões futuras, não funcionalidades já descritas no pseudocódigo original.

15. Exemplo Java integrado

Abaixo está uma versão integrada de console que reúne as operações. Salve o código como Hortifruti.java. Ela usa arrays e métodos estáticos, conserva os dados em memória e trata entradas numéricas inválidas. Para simplificar a leitura, o código está dividido em partes.

Código Java integrado — parte 1

```
import java.util.Scanner;

public class Hortifruti {
    static final int LIMITE = 100;
    static String[] nomes = new String[LIMITE];
    static String[] categorias = new String[LIMITE];
    static double[] precos = new double[LIMITE];
    static int[] quantidades = new int[LIMITE];
    static int totalProdutos = 0;
    static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int op;
        do {
            System.out.println("\\n===== HORTIFRÚTI =====");
            System.out.println("1 - Cadastrar produto");
            System.out.println("2 - Listar produtos");
            System.out.println("3 - Consultar produto");
            System.out.println("4 - Editar produto");
            System.out.println("5 - Remover produto");
            System.out.println("6 - Calcular valor do estoque");
            System.out.println("0 - Sair");
            System.out.print("Escolha uma opção: ");
            op = lerInteiro();

            switch (op) {
                case 1 -> cadastrarProduto();
                case 2 -> listarProdutos();
                case 3 -> consultarProduto();
                case 4 -> editarProduto();
                case 5 -> removerProduto();
                case 6 -> calcularValorEstoque();
                case 0 -> System.out.println("Sistema encerrado.");
                default -> System.out.println("Opção inválida!");
            }
        } while (op != 0);
    }
```

Continuação nas páginas seguintes. Os nomes Java usam camelCase (por exemplo, cadastrarProduto) e os índices começam em zero.

Código Java integrado — parte 2

```
static int lerInteiro() {
    while (true) {
        try {
            return Integer.parseInt(sc.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.print("Digite um número inteiro válido: ");
        }
    }
}

static double lerNaoNegativo(String mensagem) {
    while (true) {
        System.out.print(mensagem);
        try {
            double v = Double.parseDouble(
                sc.nextLine().trim().replace(',', '.'));
            if (v >= 0) return v;
        } catch (NumberFormatException e) {
            // Solicita o valor novamente.
        }
        System.out.println("Digite um número igual ou maior que zero.");
    }
}

static int lerQuantidade(String mensagem) {
    while (true) {
        double v = lerNaoNegativo(mensagem);
        if (v == Math.floor(v) && v <= Integer.MAX_VALUE) return (int) v;
        System.out.println("A quantidade deve ser um número inteiro.");
    }
}

static String lerCategoria() {
    while (true) {
        System.out.println("1 - Fruta\\n2 - Legume\\n3 - Verdura");
        System.out.print("Escolha a categoria: ");
        switch (sc.nextLine().trim()) {
            case "1": return "Fruta";
```

Código Java integrado — parte 3

```
                case "2": return "Legume";
                case "3": return "Verdura";
                default: System.out.println("Categoria inválida.");
            }
        }
    }

    static int buscarProduto(String nome) {
        for (int i = 0; i < totalProdutos; i++)
            if (nomes[i].equalsIgnoreCase(nome.trim())) return i;
        return -1;
    }

    static void cadastrarProduto() {
        if (totalProdutos == LIMITE) {
            System.out.println("Limite de produtos atingido.");
            return;
        }
        System.out.print("Nome do produto: ");
        String nome = sc.nextLine().trim();
        if (nome.isEmpty()) {
            System.out.println("O nome não pode ficar vazio.");
            return;
        }
        if (buscarProduto(nome) != -1) {
            System.out.println("Já existe um produto com esse nome.");
            return;
        }
        String cat = lerCategoria();
        double preco = lerNaoNegativo("Preço do produto: ");
        int qtd = lerQuantidade("Quantidade em estoque: ");

        int pos = totalProdutos;
        nomes[pos] = nome;
        categorias[pos] = cat;
        precos[pos] = preco;
        quantidades[pos] = qtd;
        totalProdutos++;
```

Código Java integrado — parte 4

```
        System.out.println("Produto cadastrado com sucesso.");
    }

    static void listarProdutos() {
        if (totalProdutos == 0) {
            System.out.println("Não há produtos cadastrados.");
            return;
        }
        for (int i = 0; i < totalProdutos; i++) {
            System.out.println("Nome: " + nomes[i]);
            System.out.println("Categoria: " + categorias[i]);
            System.out.printf("Preço: R$ %.2f%n", precos[i]);
            System.out.println("Quantidade: " + quantidades[i]);
            System.out.println("-------------------------");
        }
    }

    static void consultarProduto() {
        System.out.print("Digite o nome do produto: ");
        int pos = buscarProduto(sc.nextLine().trim());
        if (pos == -1) {
            System.out.println("Produto não encontrado.");
            return;
        }
        System.out.println("Nome: " + nomes[pos]);
        System.out.println("Categoria: " + categorias[pos]);
        System.out.printf("Preço: R$ %.2f%n", precos[pos]);
        System.out.println("Quantidade: " + quantidades[pos]);
    }

    static void editarProduto() {
        System.out.print("Produto que deseja editar: ");
        int pos = buscarProduto(sc.nextLine().trim());
        if (pos == -1) {
            System.out.println("Produto não encontrado.");
            return;
        }
        System.out.print("Novo nome: ");
```

Código Java integrado — parte 5

```
        String novoNome = sc.nextLine().trim();
        if (novoNome.isEmpty()) {
            System.out.println("O nome não pode ficar vazio.");
            return;
        }
        int outraPos = buscarProduto(novoNome);
        if (outraPos != -1 && outraPos != pos) {
            System.out.println("Já existe outro produto com esse nome.");
            return;
        }
        String cat = lerCategoria();
        double preco = lerNaoNegativo("Novo preço: ");
        int qtd = lerQuantidade("Nova quantidade: ");
        nomes[pos] = novoNome;
        categorias[pos] = cat;
        precos[pos] = preco;
        quantidades[pos] = qtd;
        System.out.println("Produto atualizado com sucesso.");
    }

    static void removerProduto() {
        System.out.print("Produto que deseja remover: ");
        int pos = buscarProduto(sc.nextLine().trim());
        if (pos == -1) {
            System.out.println("Produto não encontrado.");
            return;
        }
        for (int i = pos; i < totalProdutos - 1; i++) {
            nomes[i] = nomes[i + 1];
            categorias[i] = categorias[i + 1];
            precos[i] = precos[i + 1];
            quantidades[i] = quantidades[i + 1];
        }
        totalProdutos--;
        System.out.println("Produto removido com sucesso.");
    }

    static void calcularValorEstoque() {
```

Código Java integrado — parte 6

```
        double total = 0;
        for (int i = 0; i < totalProdutos; i++)
            total += precos[i] * quantidades[i];
        System.out.printf("Valor total do estoque: R$ %.2f%n", total);
    }
}
```